nsconv 0.1.0-rc1 · October 2026

nsconv Test Kit

Test release 0.1.0-rc1 end to end: simulated NetStream, FlowLink and a test PCE, with an evidence bundle.

Unofficial community tool. nsconv is not an Illumio product and is not supported by Illumio. Provided as is, without warranty of any kind (MIT licence).

Source and releases: github.com/roschereric/netstream-converter

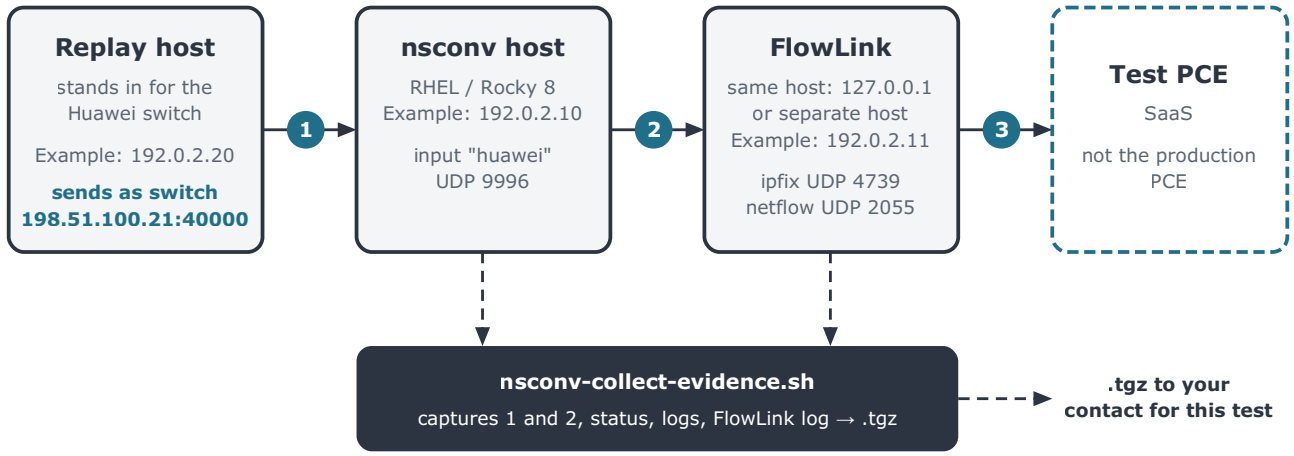
Contents

Overview 3
Install and configure the lab 5
Debug mode and where to look 7
Simulate NetStream 9
What to test 12
Collect the evidence 14
How the evidence is validated 15
Appendix: generator self-check 16

Overview

nsconv receives Huawei NetStream (NetFlow v5, v9 and IPFIX with Huawei specifics) and re-exports clean IPFIX, NetFlow v9 or NetFlow v5 to Illumio FlowLink. This kit tests the test release **0.1.0-rc1** end to end in a lab: a replay host plays recorded or generated NetStream, nsconv converts it, FlowLink sends the flows to a **test PCE**, and you send back one evidence file per host. No Huawei switch and no production PCE are needed.

Test lab and data flows (all names and addresses are examples)



#	Flow	Content
1	Replay host → nsconv, UDP 9996	NetStream v5, v9 or IPFIX, source `198.51.100.21:40000` (the example switch; 40000 is Huawei's default export source port[1]).
2	nsconv → FlowLink, UDP 4739 (IPFIX) or 2055 (v9/v5)	Clean records. With `source_address = "exporter"` (default) each datagram keeps the switch IP and port as its source, so FlowLink sees the switches themselves.
3	FlowLink → test PCE, HTTPS	Aggregated flow summaries, every `aggregation_minutes` (default 10, minimum 5)[2].

What you need

- **nsconv host**: RHEL, Rocky or Alma Linux 8 x86_64 (9 also works), root access, `tcpdump`.
- **Replay host**: any Linux VM on the **same Layer 2 segment** as the nsconv host, with Python 3 and `tcpreplay` (EPEL 8 and 9 ship tcpreplay 4.6.1[3]).
- **FlowLink 2.0.1** on the nsconv host or on a separate host of the same segment, paired with a **test PCE** (an API key of that PCE).
- The three kit files: this guide, nsconv-sample-pcap.py (writes Huawei-style NetStream test captures with synthetic data) and nsconv-collect-evidence.sh (collects the evidence).

SOURCES

1. [official-Huawei, YANG] huawei-netstream module, source-ipv4-port default 40000 — netconfcentral.org/modules/huawei-netstream/2019-06-27
2. [official-Illumio] Configure Flowlink — product-docs-repo.illumio.com/Tech-Docs/Core/26.1/Install-Upgrade-Admin/out/en/flowlink-2-0-0-configuration-and-usage/configure-flowlink.html
3. [official-Fedora] tcpreplay package (EPEL 8, EPEL 9) — packages.fedoraproject.org/pkgs/tcpreplay/tcpreplay/

Install and configure the lab

STEP 1 **Install nsconv 0.1.0-rc1 on the nsconv host**

Download the RPM and `SHA256SUMS` from the `releases/0.1.0-rc1/` folder of github.com/roschereric/netstream-converter, check them, and install. The RPM version is `0.1.0~rc1` (RPM cannot use "-" in a version; the final 0.1.0 upgrades it).

NSCONV HOST

```
# Download (or save both files from the folder in a browser)
BASE=https://github.com/roschereric/netstream-converter/raw/main/releases/0.1.0-rc1
curl -LO $BASE/nsconv-0.1.0~rc1-1.x86_64.rpm -LO $BASE/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS      # must print: nsconv-0.1.0~rc1-1.x86_64.rpm: OK
sudo dnf install ./nsconv-0.1.0~rc1-1.x86_64.rpm
nsconvctl version
```

The RPM installs `/usr/bin/nsconvctl`, the unit `nsconv.service` (user `nsconv`, only capability `CAP_NET_RAW`), the config `/etc/nsconv/nsconv.toml` and a sysctl file that raises `net.core.rmem_max` to 8 MiB when it is lower. The service is not started yet.

STEP 2 **Minimal nsconv configuration for the lab**

The defaults already fit the lab: input `huawei` on UDP 9996, one IPFIX output to `127.0.0.1:4739` sent from the switch address (`source_address = "exporter"`). Only the allow-list must be set: it is empty by default, so everything is dropped. `nsconvctl config set` keeps the comments, validates, writes and reloads.

NSCONV HOST

```
# Allow the example switches (the replay host sends with these source IPs)
sudo nsconvctl config set input.huawei.allow '["198.51.100.0/24"]'
# FlowLink on a separate host: point the output at it (example 192.0.2.11)
sudo nsconvctl config set output.flowlink.target 192.0.2.11:4739
# Open the input port to the replay host, start the service
sudo firewall-cmd --permanent --add-port=9996/udp && sudo firewall-cmd --reload
sudo systemctl enable --now nsconv
sudo nsconvctl status
```

> **exporter or local**
>
> **exporter** (default): FlowLink sees each switch IP, so its `remote_addrs` lists the switch IPs, exactly as if the switches sent to FlowLink directly. FlowLink must be on the nsconv host or on the same Layer 2 segment.
> **local**: nsconv sends from its own address; `remote_addrs` is `127.0.0.1` (same host) or the nsconv host IP. Switch with `sudo nsconvctl config set output.flowlink.source_address local`.

STEP 3 **FlowLink against the test PCE**

Install FlowLink 2.0.1 as described in the FlowLink guide, store the **test PCE** API key and secret in `/etc/illumio-flowlink/api_info`[1], and use one consumer per protocol. FlowLink has no default ports and listens on the ports you configure[2]. Never reuse a FlowLink port for the nsconv input.

/ETC/ILLUMIO-FLOWLINK/CONFIG.YAML (FLOWLINK HOST)

```
pce_addr: test-pce.example.com:443          # example: your TEST PCE
org_id: 1                                   # example: org of the test PCE (SaaS)
api_key: $cat /etc/illumio-flowlink/api_info
data_directory: /var/lib/illumio-flowlink/data
aggregation_minutes: 5                      # minimum: flows reach the PCE sooner
consumers:
  - name: nsconv-ipfix                      # nsconv output protocol = "ipfix"
    parser:
      type: ipfix
    connectors:
      - type: udp
        properties:
          ports: '4739'
          remote_addrs: ['198.51.100.21', '198.51.100.22']   # exporter mode: the switch IPs
  - name: nsconv-netflow                    # protocol = "netflow9" or "netflow5"
    parser:
      type: netflow
    connectors:
      - type: udp
        properties:
          ports: '2055'
          remote_addrs: ['198.51.100.21', '198.51.100.22']
```

FLOWLINK HOST

```
sudo systemctl restart illumio-flowlink
sudo -u ilo-flowlink /usr/local/bin/illumio-flowlink-ctl status
ss -lunp | grep flowlink                  # 4739 and 2055 listening
# Separate FlowLink host only: open its ports to the nsconv host
sudo firewall-cmd --permanent --add-port=4739/udp --add-port=2055/udp && sudo firewall-cmd --
reload
```

`remote_addrs` takes single IPs, not CIDRs[2]. With `source_address = "local"` use `'127.0.0.1'` (same host) or the nsconv host IP (example `'192.0.2.10'`). In the PCE, flow IPs show in Illumination only when they belong to workloads[3]: if the example workload IPs do not appear, create unmanaged workloads for a few of them in the test PCE.

SOURCES

1. [official-Illumio] Deploy Flowlink as a Linux service — product-docs-repo.illumio.com/…/configure-flowlink/deploy-flowlink-as-a-linux-service.html
2. [official-Illumio] Ingested flow examples (consumers, ports, remote_addrs) — product-docs-repo.illumio.com/…/configure-flowlink/ingested-flow-examples.html
3. [official-Illumio] Scale and limitations — product-docs-repo.illumio.com/…/overview/scale-and-limitations.html

Debug mode and where to look

nsconv

The log level is the config key `log.level` (`error`, `warn`, `info`, `debug`); there is no separate flag. Changing it reloads the daemon without a restart. At `debug`, every dropped, clamped or defaulted datagram or flow logs one line per event kind, at most 5 lines per 10 s per event and exporter and 2000 lines per 10 s in total ("N similar suppressed").

NSCONV HOST

```
sudo nsconvctl config set log.level debug      # back with: ... log.level info
nsconvctl logs -f                             # follow the journal of nsconv
nsconvctl logs --since 30m                     # or "2026-10-21 14:00"
sudo nsconvctl logs --recent 200               # the daemon's in-memory ring
journalctl -u nsconv NSCONV_EXPORTER=198.51.100.21:40000 -o verbose   # one switch
sudo nsconvctl status --detail                 # health, alerts, exporters, outputs, counters
sudo nsconvctl status --exporter 198.51.100.21  # one switch: sampling, skew, sequence loss
sudo nsconvctl --json status --detail          # the same as JSON (accounting.unaccounted must be
0)
sudo nsconvctl tui                             # full-screen view, Alerts tab; --lang es
```

Warnings have stable ids (`sampling-default`, `restamped`, `no-template`, `seq-gap`, `collector-down`, ...). Silence one with `sudo nsconvctl dismiss <id>`; it returns if its value doubles.

Debug collector and raw captures

`nsconvctl decode --listen` is a small collector that prints every flow it receives. Use it on a port that FlowLink does not use (example 9997).

NSCONV HOST

```
# a) What the replay host sends, before nsconv: replay to port 9997 instead of 9996
nsconvctl decode --listen 0.0.0.0:9997
# b) What nsconv sends: add a second output to /etc/nsconv/nsconv.toml, then: sudo nsconvctl
reload
#    [[output]]
#    name = "debug"
#    protocol = "ipfix"
#    target = "127.0.0.1:9997"
#    source_address = "local"
nsconvctl decode --listen 127.0.0.1:9997 --count 50
# c) Raw NetStream datagrams of one switch into a pcap (runs tcpdump, default 60 s / 2000
datagrams)
sudo nsconvctl debug-capture 198.51.100.21 --secs 120
# d) Decode any capture offline
nsconvctl decode nsconv-capture-198.51.100.21-*.pcap
```

FlowLink

FlowLink logs to `/var/log/illumio-flowlink/`. For verbose output, stop the service and start it in verbose mode as `ilo-flowlink`[1]; debug output shows incoming records, the IP, port and protocol of each record, every aggregation and upload, and the PCE response code[2]. Use it only during the test, then return to the service.

FLOWLINK HOST

```
sudo -u ilo-flowlink /usr/local/bin/illumio-flowlink-ctl stop
sudo -u ilo-flowlink /usr/local/bin/illumio-flowlink-ctl start -v
sudo tail -f /var/log/illumio-flowlink/flowlink.log
# after the test
sudo -u ilo-flowlink /usr/local/bin/illumio-flowlink-ctl stop && sudo systemctl start illumio-flowlink
```

SOURCES

1. [official-Illumio] Deploy Flowlink as a Linux service (log directory, start -v) — product-docs-repo.illumio.com/…/deploy-flowlink-as-a-linux-service.html
2. [official-Illumio] Flowlink troubleshooting (debug output) — product-docs-repo.illumio.com/…/flowlink-usage/troubleshooting.html
3. [official-systemd] journalctl(1), field matches — freedesktop.org/software/systemd/man/latest/journalctl.html

Simulate NetStream

Generate a test capture

`nsconv-sample-pcap.py` writes a pcap of Huawei-style NetStream datagrams: NetFlow v5 (48-byte records), v9 (Huawei original-flow template layout, with the Huawei private field 232) or IPFIX (relative times plus IE 160 in every record, Huawei enterprise fields), UDP source port 40000, templates in their own datagrams before the data and repeated every 10 data datagrams. It needs only Python 3.6 or later (RHEL 8: `/usr/libexec/platform-python`). All values are synthetic.

REPLAY HOST

```
curl -LO https://roschereric.github.io/netstream-converter/test-kit/nsconv-sample-pcap.py
# MAC of the nsconv host NIC (run on the nsconv host): ip -br link show <nic>
python3 nsconv-sample-pcap.py --version 9 --switch 198.51.100.21 --dst 192.0.2.10 \
    --dst-mac 52:54:00:12:34:56 --flows 200 --out v9.pcap --expect v9.expect.jsonl
```

Option	Meaning (default)
`--version 5|9|ipfix`	Export version (9).
`--switch IP[,IP…]`	Switch source address(es); each switch gets its own flows (198.51.100.21).
`--sport` / `--dst` / `--dport`	Switch source port (40000), nsconv host IP (192.0.2.10), nsconv input port (9996).
`--dst-mac` / `--src-mac`	Frame MACs. `--dst-mac` must be the nsconv NIC for tcpreplay.
`--src` / `--dst-hosts` / `--dst-ports`	Workload IPs or CIDRs and ports (10.1.1.0/24 → 10.2.2.0/24, ports 443,22,3306,8080,53; IPv6: 2001:db8:a::/112 → 2001:db8:b::/112).
`--flows N` / `--ipv6`	Flows per switch (200) / IPv6 workloads (v9 and IPFIX only).
`--sampling N`	1-in-N rate written as IE 34 in every v9/IPFIX record (0 = not reported, as many Huawei switches do).
`--domain N`	v9 Source ID / IPFIX Observation Domain / v5 engine id (0): a second value emulates a second line card.
`--pps` / `--template-every` / `--data-only`	Datagram rate in the pcap timestamps (100); template repeat (10); data without templates (restart test).
`--expect FILE` / `--check`	Write the generated flows as JSON lines / decode the pcap with `nsconvctl` and compare every flow.

> **Generate just before you replay**
>
> Flow times are set at generation time (they end 1 to 30 s before it). A pcap replayed more than 5 minutes later is outside `convert.max_clock_skew_secs = 300`: nsconv then shifts the times to the receive time and counts `restamped`. Keep the clocks of all hosts synchronized (chrony).

Option A: `nsconvctl replay` (simple)

Sends the NetStream payloads of a capture with a normal UDP socket. It never spoofs: nsconv and FlowLink see the **replay host** address (example 192.0.2.20) and an ephemeral source port. Good for a first check; for the switch identity tests use option B.

REPLAY HOST AND NSCONV HOST

```
# nsconv host: allow the replay host; FlowLink remote_addrs then lists 192.0.2.20 (exporter mode)
sudo nsconvctl config set input.huawei.allow '["198.51.100.0/24", "192.0.2.20"]'
# replay host: copy nsconvctl from the RPM or the release binary, then
nsconvctl replay v9.pcap --to 192.0.2.10:9996 --rate 200
```

Option B: tcpreplay (keeps the switch source address)

tcpreplay sends the frames exactly as they are in the pcap, so the datagrams arrive from `198.51.100.21:40000`: the allow-list, the per-switch view and `source_address = "exporter"` behave as with a real switch. The generator already writes the right destination; for any other capture (for example one taken with `debug-capture`, which is a Linux “cooked” capture) rewrite it first with tcprewrite.

REPLAY HOST

```
sudo dnf install epel-release && sudo dnf install tcpreplay   # Rocky/Alma; RHEL: enable EPEL
first
# Only for captures not made by the generator: old collector IP (example 192.0.2.50) to the
# nsconv host, its MAC, recalculated checksums. Add --dlt=enet for a cooked capture (debug-
capture).
tcprewrite --infile=capture.pcap --outfile=lab.pcap \
    --enet-dmac=52:54:00:12:34:56 --enet-smac=02:00:00:00:00:01 \
    --dstipmap=192.0.2.50/32:192.0.2.10/32 --fixcsum
# Replay on the NIC that faces the nsconv host
sudo tcpreplay --intf1=eth0 v9.pcap                          # pcap timing (100 datagrams/s)
sudo tcpreplay --intf1=eth0 --pps=2000 --loop=30 burst.pcap   # load test
```

tcpreplay / tcprewrite option	Effect (official manual)
`--intf1` (`-i`)	Output interface; required.
`--pps` / `--mbps` / `--topspeed`	Packets per second / Mbps / as fast as possible; default is the pcap timing.
`--loop` / `--duration` / `--limit`	Repeat the file N times / stop after N seconds / after N packets.
`--dstipmap` (`-D`)	Pseudo-NAT of destination IPs, `CIDR:CIDR` pairs.
`--enet-dmac` / `--enet-smac`	Override destination / source MAC.
`--dlt=enet`	Convert a Linux cooked capture to Ethernet (needs both MACs).
`--fixcsum` (`-C`)	Recalculate IPv4/TCP/UDP checksums.

Do not use `--unique-ip`: it changes the IP header (the switch address), not the addresses inside the NetStream records.

> **Replay from a separate host, never from the nsconv host**
>
> tcpreplay writes frames straight to the NIC's transmit side. The host's own network stack never receives what it transmits, and the switch port does not send a frame back out of the port it came from, so a replay on the nsconv host itself never reaches nsconv. Use a second VM or host on the same Layer 2 segment (same VLAN, no router in between).

Network checks on the nsconv host

- **Reverse-path filter**: RHEL's default `rp_filter = 1` (strict) drops a datagram whose source (198.51.100.21) would be routed back through a different interface. The default route through the receiving NIC passes. If the nsconv host has several NICs, add a route for the switch range through the receiving NIC, or set loose mode on it: `sudo sysctl -w net.ipv4.conf.<nic>.rp_filter=2`. Dropped datagrams show in `nstat -az TcpExtIPReversePathFilter`, not in nsconv.
- **firewalld**: UDP 9996 open on the zone of the receiving NIC (step 2).
- **Allow-list**: `input.huawei.allow` holds the switch range (option B) or the replay host IP (option A); others count as `denied`.
- **Check arrival**: `sudo tcpdump -ni any udp port 9996 -c 5`, then `sudo nsconvctl status`.

SOURCES

1. [official-tcpreplay] tcpreplay(1) manual — tcpreplay.appneta.com/reference/man/tcpreplay/
2. [official-tcpreplay] tcprewrite(1) manual — tcpreplay.appneta.com/reference/man/tcprewrite/
3. [official-tcpreplay] tcprewrite overview (DLT conversion) — tcpreplay.appneta.com/tools/tcprewrite/
4. [official-kernel] ip-sysctl: rp_filter 0/1/2 — docs.kernel.org/networking/ip-sysctl.html
5. [lab] Rocky 8.10 test VM, 2026-10-20: frames with a source outside the link's routes were dropped by rp_filter (counter TcpExtIPReversePathFilter), and arrived once a route back through the receiving interface existed.

What to test

Run the evidence script (next section) around each test or group of tests and write the test numbers in `--note`. "Status" means `sudo nsconvctl status --detail`; "Explorer" means the test PCE's Explorer, filtered by the example workload IPs. Flows reach the PCE after the FlowLink aggregation interval.

#	Test	How	Expected	Where
1	Install and sandbox	Step 1–2; `systemd-analyze security nsconv`	Service active; exposure "OK"; process user `nsconv` with only CAP_NET_RAW	Evidence: nsconv-security, nsconv-process
2	v9 IPv4 → IPFIX (baseline)	Generate `--version 9 --flows 200`, tcpreplay	`flows_in` = `flows_out` = 200, `unaccounted` 0; FlowLink accepts 198.51.100.21; flows in Explorer	Status, FlowLink log, Explorer
3	v5 input	`--version 5`	Exporter shows v5; rate "default 1000" and alert `sampling-default` (the v5 header carries no rate, as in the Huawei captures we know)	Status, Explorer
4	IPFIX input	`--version ipfix`	Same flows; Huawei enterprise fields ignored: `malformed`, `template_rejects` 0	Status, Explorer
5	Output v9 and v5	`config set output.flowlink.protocol netflow9` and `target …:2055`; then `netflow5`	Flows in Explorer through the netflow consumer; v5 output counts IPv6 flows as `v6_on_v5` and does not send them	Status, FlowLink log, Explorer
6	Switch identity (exporter)	`--switch 198.51.100.21,198.51.100.22`; then remove .22 from `remote_addrs`	Both IPs accepted and listed by FlowLink; after the removal FlowLink discards .22 only	FlowLink log, Explorer
7	local mode	`source_address local`, `remote_addrs` = `127.0.0.1` or the nsconv IP	Flows accepted; note how FlowLink's log names a 127.0.0.1 exporter	FlowLink log
8	Two line cards, one IP	Two pcaps, same `--switch`, `--domain 1` / `--domain 2`, different `--src`	Two sessions in nsconv; both workload sets in Explorer (FlowLink keeps the domains apart)	Status, Explorer
9	Sampling (IE 34)	`--sampling 2048`; then `config set input.huawei.sampling_override '{ "198.51.100.21" = 1000 }'`	Rate 2048 "reported", then 1000 "override", carried in every output record; note whether the PCE scales counts	Status, output pcap, Explorer
10	Bytes (IE 1 and 85)	`--flows 1 --expect one.jsonl`	Explorer shows the byte count of `one.jsonl`	Explorer
11	IPv6	`--ipv6` with v9 and with IPFIX	IPv6 flows decoded and forwarded; note whether they show in Explorer	Status, Explorer

12	Time buckets	Generate and replay at once; note the UTC time	Flows in the interval of the replay time; `restamped` 0	Explorer, status
13	Clock skew	Replay a pcap generated over 10 min earlier	`restamped` grows, alert `restamped` ; flows dated at receive time	Status, Explorer
14	Reload	`config set log.level debug` , then `info`	Same PID, “reload” in the log, flows continue	Logs, status
15	Restart, templates kept	Replay v9; `systemctl restart nsconv` ; replay a `--data-only` pcap of the same switch	Data-only flows decoded (`no_template` 0): templates were restored from `/var/lib/nsconv`	Status
16	Collector down	Same host: `systemctl stop illumio-flowlink` during a replay, then start it	nsconv stays up; alert `collector-down` within about 15 s; flows resume after the next template refresh (20 s)	Status, logs
17	Burst	`--flows 50000 --pps 5000` , `tcpreplay --pps=5000 --loop=10`	No kernel drops (`RcvbufErrors` unchanged), `unaccounted` 0, `flows_out` = `flows_in`	Evidence: udp-snmp, status
18	FlowLink listener	`ss -lunp | grep flowlink`	Note IPv4, IPv6 or dual-stack bind; the evidence copies FlowLink’s config schema	Evidence: ss-udp
19	SELinux, firewall	Whole test	No AVC denial for nsconv or FlowLink	Evidence: avc-window, selinux

Answers we are looking for

Tests 6–11 and 18 tell us how FlowLink 2.0.1 treats switch-IP sources from a co-located relay, line-card domains, the sampling rate, the byte counters (IE 1 or 85), IPv6 and time buckets. Write what you see in the PCE next to the test number in `--note` , or in your message with the file.

SOURCES

1. [nsconv] counters, alerts and commands as documented in the release (nsconvctl --help, operator guide) — github.com/roschereric/netstream-converter
2. [official-Illumio] Flowlink scale and limitations — product-docs-repo.illumio.com/…/overview/scale-and-limitations.html

Collect the evidence

`nsconv-collect-evidence.sh` runs as root for a time window (default 300 s). It captures the NetStream input (switch → nsconv) and every nsconv output (nsconv → collector) with tcpdump, each file capped at 100 MB, and collects everything needed to check the run. Nothing on the host is changed; a missing tool or file is recorded and the run goes on.

NSCONV HOST

```
curl -LO https://roschereric.github.io/netstream-converter/test-kit/nsconv-collect-evidence.sh
sudo bash nsconv-collect-evidence.sh --dry-run            # shows what it will run
sudo bash nsconv-collect-evidence.sh --secs 300 --sample v9.pcap --note "tests 2, 6: v9, two
switches"
# now start the replay on the replay host; it must finish inside the window
# FlowLink on a separate host: run there at the same time
sudo bash nsconv-collect-evidence.sh --flowlink-only --secs 300
```

In the .tgz	Content
`pcap/`	`input.pcap`, `output-<name>.pcap` (FlowLink host: `flowlink-input.pcap`), each decoded by `nsconvctl decode` into `.flows.jsonl`
`cmd/`	`nsconvctl version`, `status --json --detail` before and after, `logs --since`, journal (JSON), `config show`, unit status, `systemd-analyze security`, rp_filter and buffer sysctls, firewalld, nftables, SELinux and AVC denials, `ss -lunp`, `ip addr/route`, clock (chrony), FlowLink version, service and `illumio-flowlink-ctl status`
`files/`	`nsconv.toml`, FlowLink `config.yaml` with secrets replaced by `<REDACTED>`, the FlowLink log lines of the window, FlowLink's config schema, your sample pcaps
`MANIFEST.tsv`, `SHA256SUMS`, `SUMMARY.txt`, `NOTE.txt`	Every item with its exit code and SHA-256; checksums for `sha256sum -c`; flows per capture and `unaccounted`; your note

Send it back

The script prints the file name `nsconv-evidence-<host>-<UTC time>.tgz` and its SHA-256. Send the .tgz (and the one from the FlowLink host, if separate) with the SHA-256 and your notes through your usual channel to your contact for this test. The FlowLink API key file is never read, and API keys, secrets, tokens and passwords in `config.yaml` are replaced; the captures contain only the synthetic test flows.

SOURCES

1. [official-tcpdump] tcpdump(1): -c, -U, -w, -i any — tcpdump.org/manpages/tcpdump.1.html
2. [lab] Rocky 8.10 test VM, 2026-10-20: the script ran as root (nsconv in exporter mode, collector on 127.0.0.1) and in `--flowlink-only` mode with a sample FlowLink config; secrets were redacted and every item was listed in MANIFEST.tsv.

How the evidence is validated

A test passes when every step below holds for its window. The checks use only what is inside the .tgz.

- [ ] **Input = generated** — the flows in `pcap/input.flows.jsonl` equal the generated flows (`--expect` file, or the sample pcap decoded again).
- [ ] **Output = input** — `pcap/output-<name>.flows.jsonl` holds the same flows (addresses, ports, protocol, packets, bytes, times), each exactly once, with the switch IP and port as exporter in exporter mode.
- [ ] **Nothing unaccounted** — in `nsconv-status-after.json`, `accounting.unaccounted` is 0, the difference of `flows_in` and `flows_out` before and after equals the replayed flows, `denied`, `malformed` and `send_errors` did not grow, and `RcvbufErrors` in `udp-snmp` did not change.
- [ ] **FlowLink accepted the switches** — `files/flowlink-window.log` names the switch IPs (or 127.0.0.1 / the nsconv host in local mode) and shows no discard or “bad data” for them.
- [ ] **The PCE received them** — the same log shows the uploads to the test PCE succeeding (with the PCE response code in verbose mode), and Explorer shows the workload flows of the test (your note).
- [ ] **Host clean** — no AVC denial, service active, no restart during the window (`NRestarts`).

SOURCES

1. [nsconv] status schema `nsconv.status/1`: accounting equations and the `unaccounted` residual, shown by `nsconvctl status`.

Appendix: generator self-check

`--check` decodes the generated pcap with `nsconvctl decode --json` and compares every flow (exporter, addresses, ports, protocol, packets, bytes, start and end time, sampling rate) with what was generated. Run it on the nsconv host before a test.

NSCONV HOST

```
/usr/libexec/platform-python nsconv-sample-pcap.py --version ipfix --ipv6 --sampling 2048 --out
t.pcap --check
# check OK: nsconvctl decoded 200 of 200 flows with identical addresses, ports, protocol, packets,
bytes and times
```

Before this release, ten variants (v5; v5 with a second engine id; v9 and IPFIX, each IPv4 and IPv6, with and without a reported sampling rate; two switches each, 666 flows per file) were generated and decoded with no difference, on macOS and on Rocky Linux 8.10 with `platform-python` 3.6. On Rocky 8.10, the frames were also replayed with the switch source address into nsconv (exporter mode): the input and output captures of the evidence script held the same 500 flows, `unaccounted` was 0, and after a restart a data-only capture was still decoded from the restored templates.

SOURCES

1. [lab] macOS and Rocky Linux 8.10 (platform-python 3.6.8), 2026-10-20: 10 of 10 generator variants decoded without difference.

Unofficial community tool. nsconv is not an Illumio product and is not supported by Illumio. Provided as is, without warranty of any kind (MIT licence).

Illumio, FlowLink and PCE are names of products of Illumio, Inc.; Huawei and NetStream belong to Huawei Technologies Co., Ltd. They are used here only to name the products nsconv works with. Validate technical details against the official documentation of your product versions.

Source and releases: github.com/roschereric/netstream-converter